

AIDOS LAB
AI FOR DATA-ORIENTED SCIENCE

UNI FR

Lecture 9: topological losses

Geometry and Topology in Machine Learning

Inés García Redondo — November 10, 2026

Recap

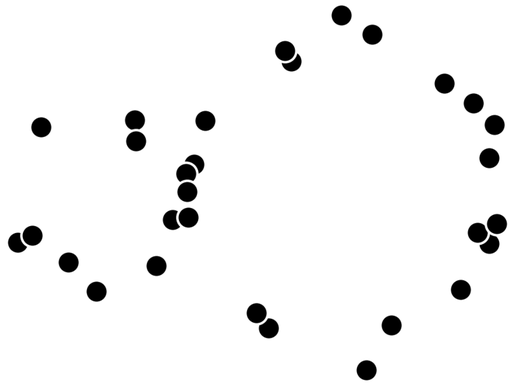
Lecture 9 — Topological losses

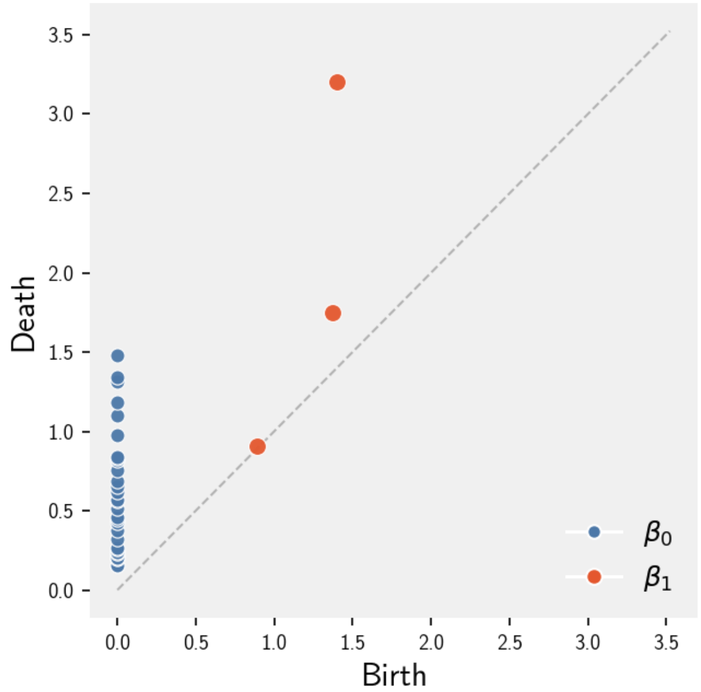
Persistent homology pipeline



Point cloud ⟶ Čech ⟶ Persistence diagram ⟶ ???

$f : K \to \mathbb{R}$ $\quad K_t = f^{-1}(-\infty, t]$

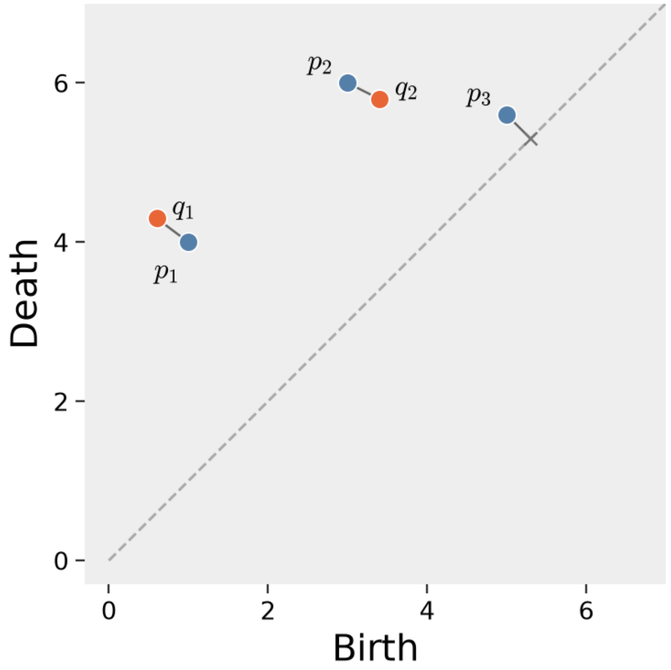




Distances between diagrams

Def. Matching between persistence diagrams

A matching between persistence diagrams is a bijection $\psi : \mathrm{PD}(X) \to \mathrm{PD}(Y)$ between the points in the diagrams, where the points in the diagonal in both diagrams are counted with infinite multiplicity



Bottleneck distance

$$d_B\left(\mathrm{PD}(X), \mathrm{PD}(Y)\right) = \inf_{\psi:\mathrm{PD}(X)\to\mathrm{PD}(Y)} \max\{\|x - \psi(x)\|_\infty : \forall x \in \mathrm{PD}(X)\}$$

cost of the matching

Wasserstein distance

$$W_p\left(\mathrm{PD}(X), \mathrm{PD}(Y)\right) = \inf_{\psi:\mathrm{PD}(X)\to\mathrm{PD}(Y)} \left(\sum_{x\in\mathrm{PD}(X)} \|x - \psi(x)\|_\infty^p \right)^{1/p}$$

$p = q$

$$W_{p,q}\left(\mathrm{PD}(X), \mathrm{PD}(Y)\right) = \inf_{\psi:\mathrm{PD}(X)\to\mathrm{PD}(Y)} \left(\sum_{x\in\mathrm{PD}(X)} d_q(x, \psi(x))^p \right)^{1/p}$$

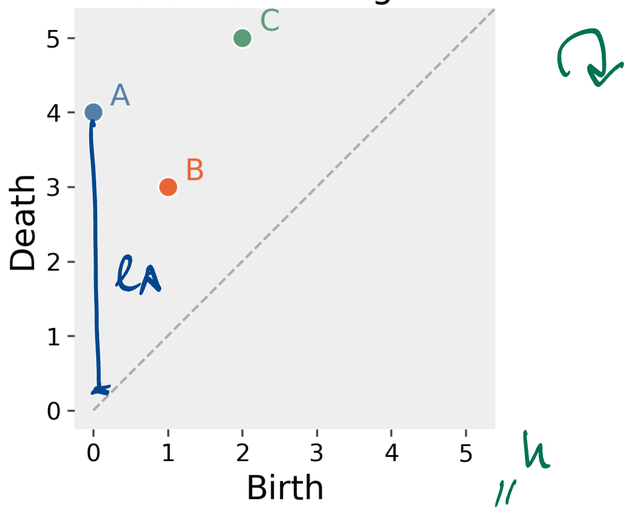
Persistent homology pipeline

vectorization



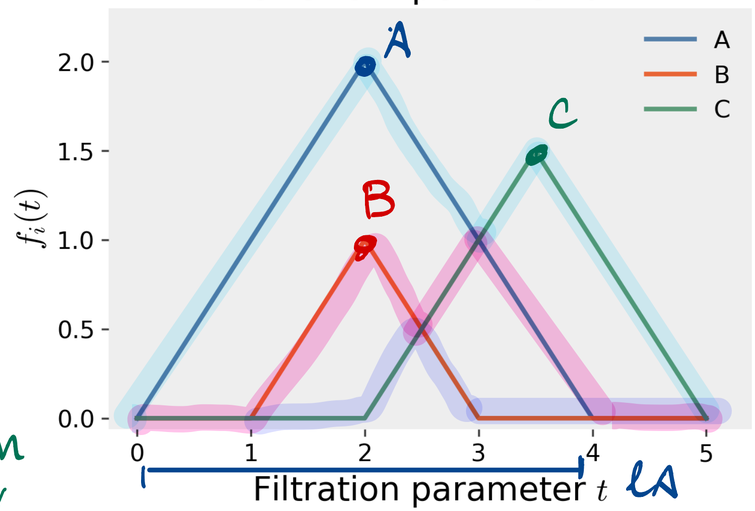
Point cloud → Čech → Persistence diagram → Persistence landscape

after change variable

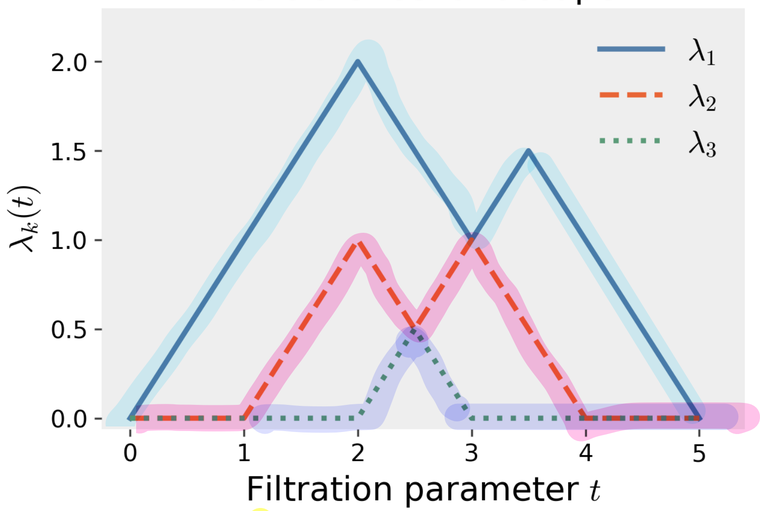


$(b, d) \mapsto \left(\frac{d-b}{2}, \frac{b+d}{2}\right)$

$h = \frac{d-b}{2}$ $m = \frac{b+d}{2}$



$f_i(t) = \max\{0, \min(t - b_i, d_i - t)\}$



$\lambda_k(t) = k\text{th largest}\{f_i(t)\}$

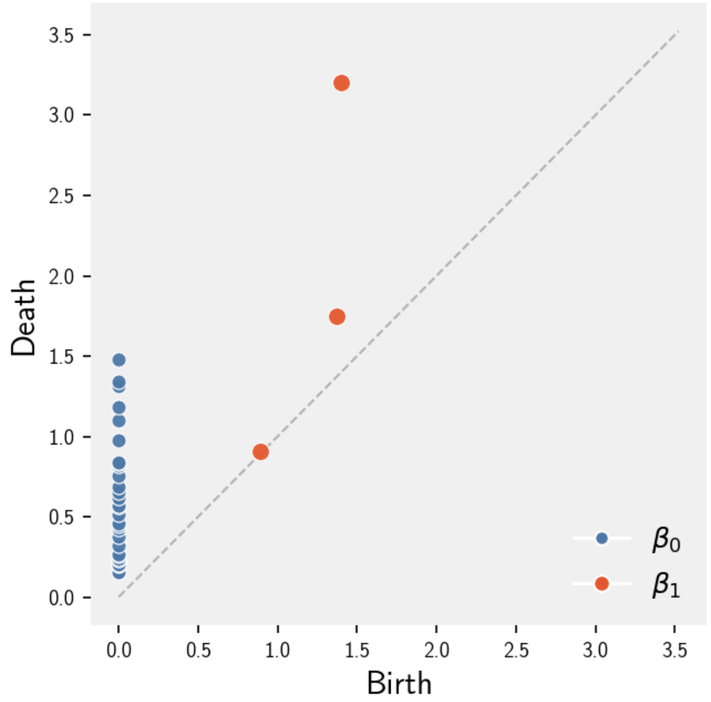
PersLay: learning from persistence diagrams

Lecture 9 — Topological losses

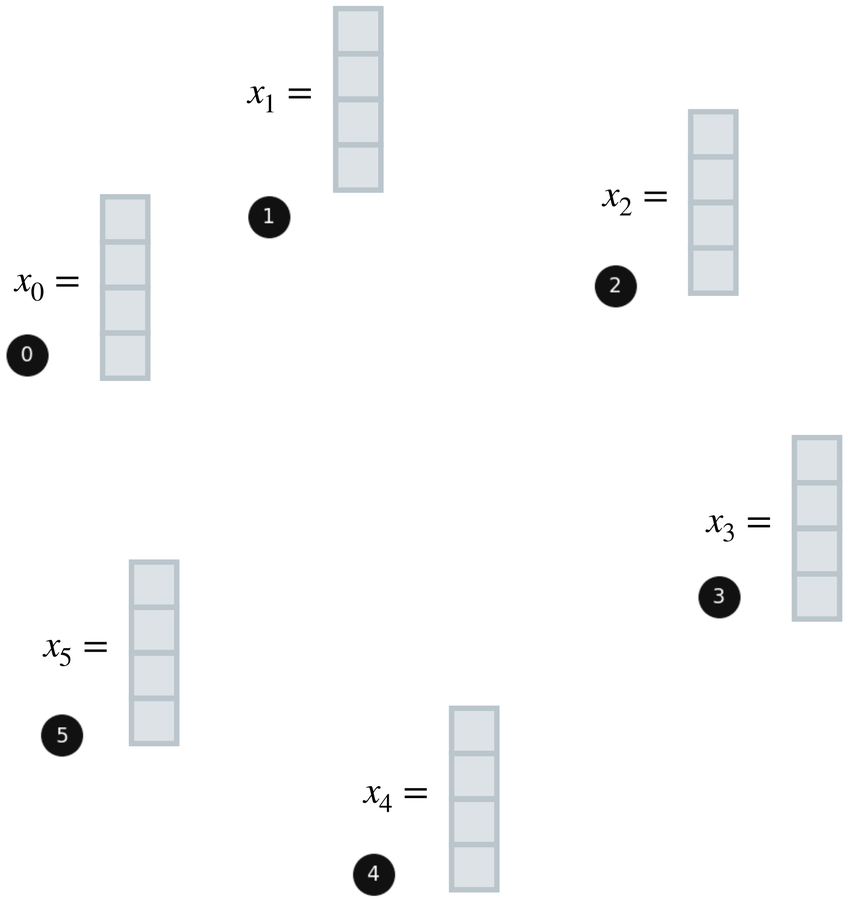
Persistent homology pipeline



Point cloud ⟶ Čech ⟶ Persistence diagram ⟶ ???



Learning from sets



- Suppose we are given a set of points S, and we want to learn a function $f : S \to \mathbb{R}^k$
- In other words, we want to learn feature vectors on the points of the set: $x_i \in \mathbb{R}^d$ for $i \in S$
- We stack them: $X = (x_1, \ldots, x_n)^\top \in \mathbb{R}^{n \times d}$

  Already specifies an order in which you are visiting the nodes

  But our function should not depend on this choice!

$$X = \begin{pmatrix} x_1^\top \\ x_2^\top \\ \vdots \\ x_n^\top \end{pmatrix}$$

Want to learn a function

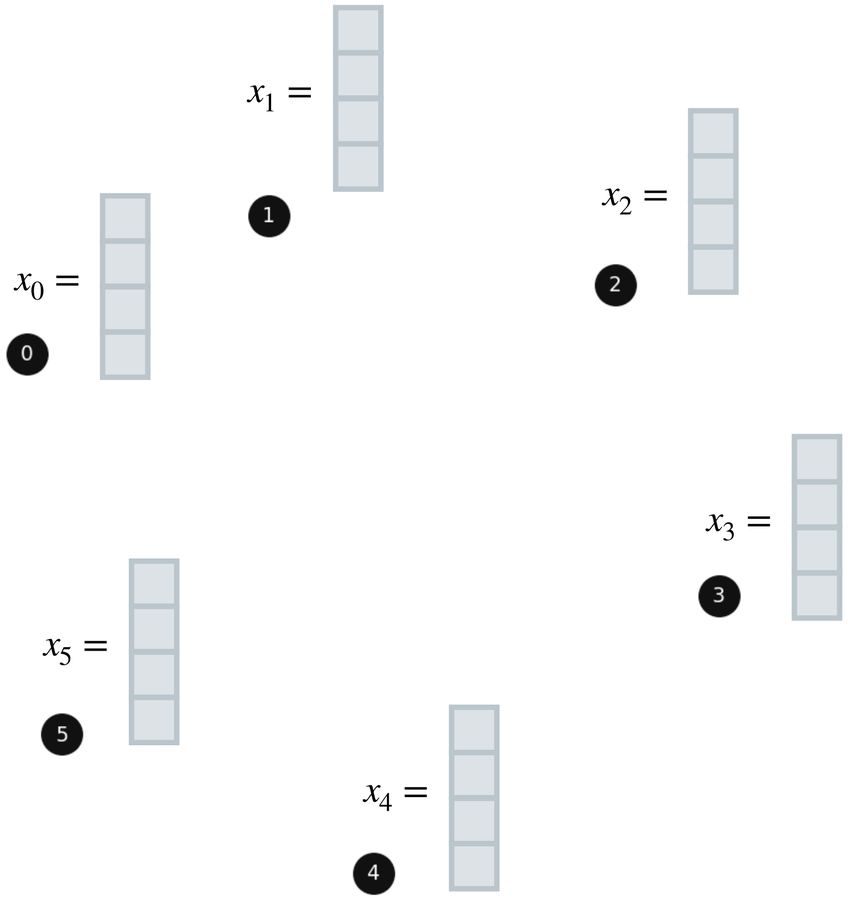
$f_\theta : X \to \mathbb{R}^k$

invariant under permutations

Invariance: $f_\theta(X) = f_\theta(PX)$ ← perm. matrix

Equivariance: $f_\theta(PX) = P f_\theta(X)$

Learning from sets



What transformations change the ordering of the nodes?

Permutations $\sigma \in S_n$

Can be represented as *permutation matrices*, which we can apply to X

Example:

$\sigma = (1\,3\,2\,4) \in S_4$

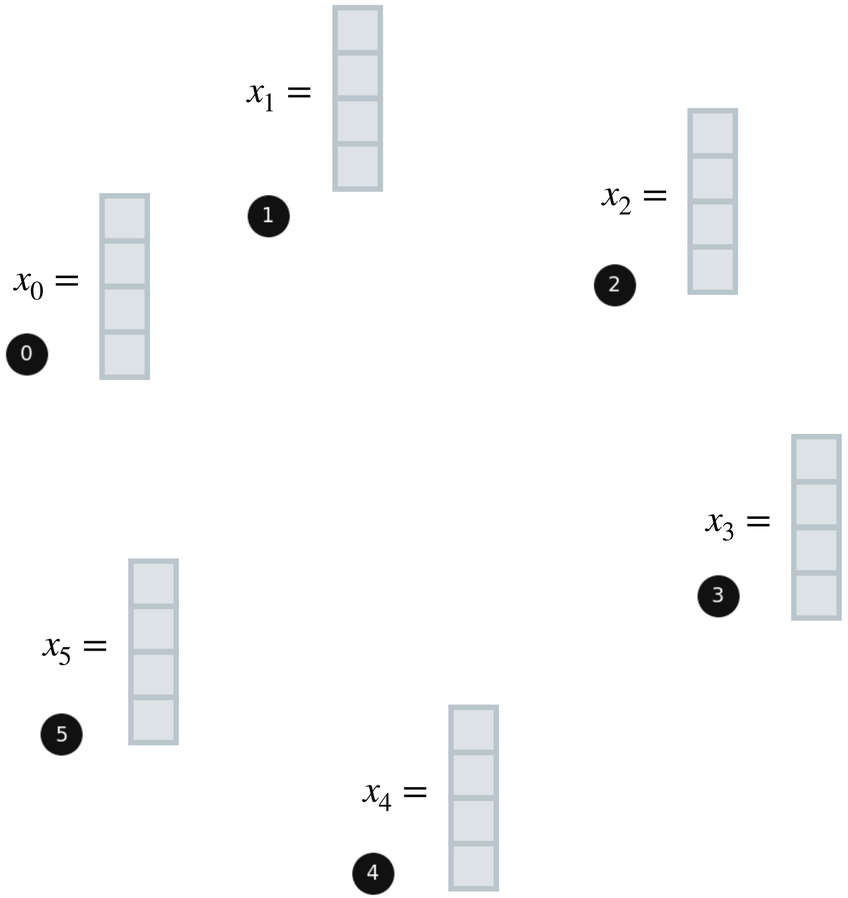
$$\sigma = \begin{pmatrix} 1 & 2 & 3 & 4 \\ 3 & 4 & 2 & 1 \end{pmatrix}$$

$$P_\sigma = \begin{pmatrix} 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 1 & 0 & 0 \\ 1 & 0 & 0 & 0 \end{pmatrix}, \quad X = \begin{pmatrix} x_1^\top \\ x_2^\top \\ x_3^\top \\ x_4^\top \end{pmatrix}, \quad P_\sigma X = \begin{pmatrix} x_3^\top \\ x_4^\top \\ x_2^\top \\ x_1^\top \end{pmatrix}$$

S_n : set of permutations of n elements

$\phi : \{1, \dots, n\} \longrightarrow \{1, \dots, n\}$ bijection

The DeepSet architecture



number of points in the set

hidden dim.

Goal: define a function $f : \mathbb{R}^{n \times d} \to \mathbb{R}^k$ taking as input

$$X = (x_1, \ldots, x_n)^\top \in \mathbb{R}^{n \times d}$$

classes

which is *permutation* invariant, that is, which for all permutation matrices P

$$f(PX) = f(X)$$

Theorem

Any such function can be written as

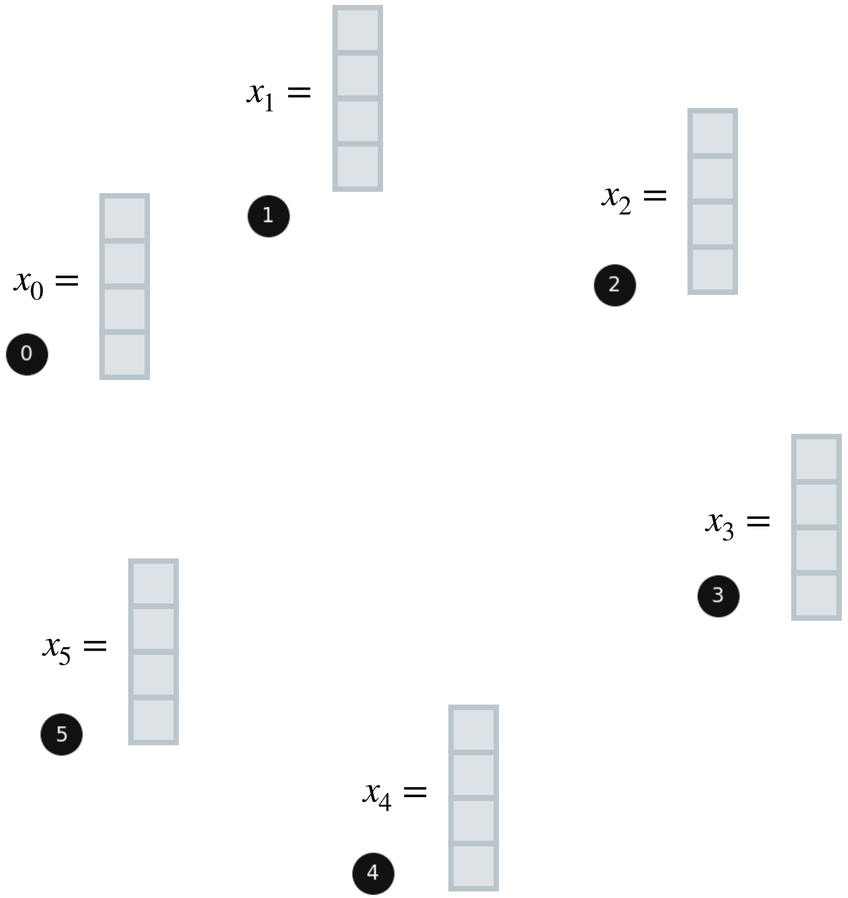
$$f(X) = \varphi\left(\sum_{i \in S} \phi(\mathbf{x}_i)\right)$$

$\in \mathbb{R}^d$

for suitable $\phi : \mathbb{R}^d \to \mathbb{R}^m, \quad \varphi : \mathbb{R}^m \to \mathbb{R}^k$

[1]Zaheer, M., Kottur, S., Ravanbakhsh, S., Poczos, B., Salakhutdinov, R. R., and Smola, A. (2017). Deep Sets. *Advances in Neural Information Processing Systems*, *30*.

The DeepSet architecture



DeepSets are the architecture built in this manner, to learn from sets

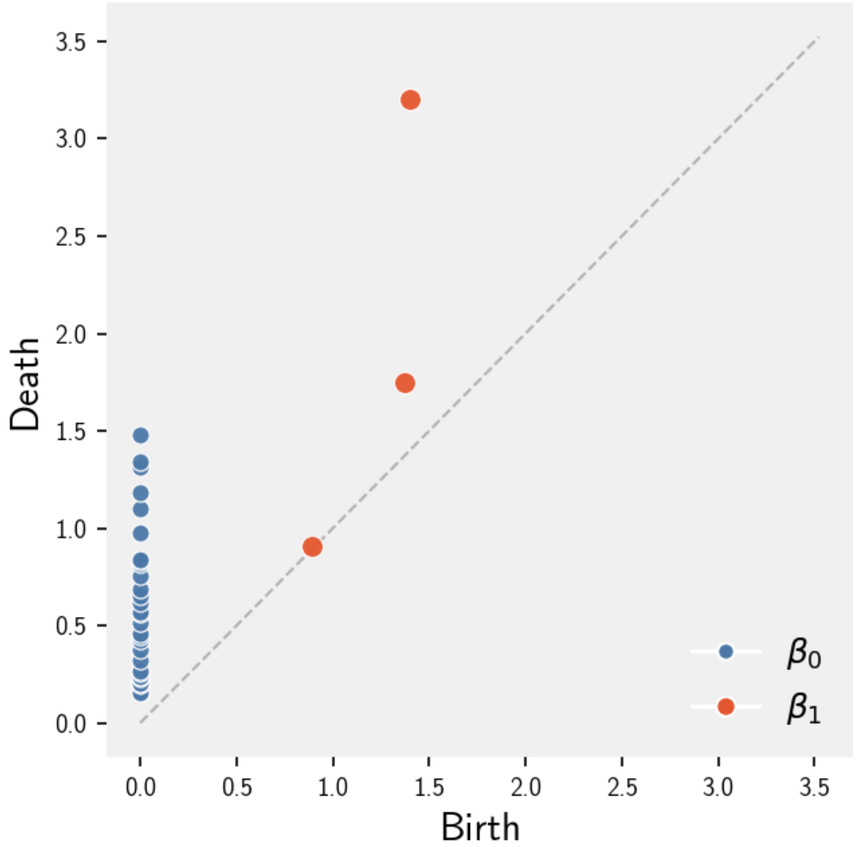
$$f(X) = \varphi_{\theta_2}\left(\sum_{i \in S} \phi_{\theta_1}(\mathbf{x}_i)\right)$$

$$\phi_{\theta_1} : \mathbb{R}^d \to \mathbb{R}^m, \quad \varphi_{\theta_2} : \mathbb{R}^m \to \mathbb{R}^k \qquad \text{MLPs}$$

Substituting the sum by any other permutation invariant operator like taking the maximum, the kth maximum, or an average, you still obtain a permutation invariant function

[1]Zaheer, M., Kottur, S., Ravanbakhsh, S., Poczos, B., Salakhutdinov, R. R., and Smola, A. (2017). Deep Sets. *Advances in Neural Information Processing Systems*, *30*.

PersLay



Extend the DeepSet architecture to accommodate persistence diagrams

$$\mathrm{PersLay}(\mathrm{PD}) := \bigoplus_{p \in \mathrm{PD}} w_{\theta_1}(p) \cdot \varphi_{\theta_2}(p)$$

With $\bigoplus$ a permutation invariant operator, $w_{\theta_1} : \mathbb{R}^2 \to \mathbb{R}$ a weight function for the points in the diagram and $\varphi_{\theta_2} : \mathbb{R}^2 \to \mathbb{R}^q$ a *point transformation* (a vectorization)

This general formulation allows to recover several popular vectorizations

Carrière, M., Chazal, F., Ike, Y., Lacombe, T., Royer, M., and Umeda, Y. (2020). PersLay: A Neural Network Layer for Persistence Diagrams and New Graph Topological Signatures. *Proceedings of the Twenty Third International Conference on Artificial Intelligence and Statistics*, 2020.

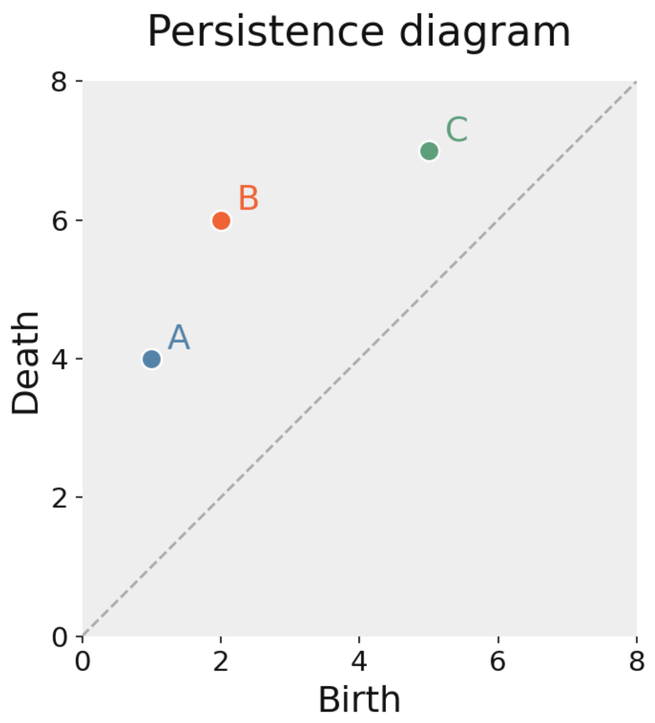
Point transformations

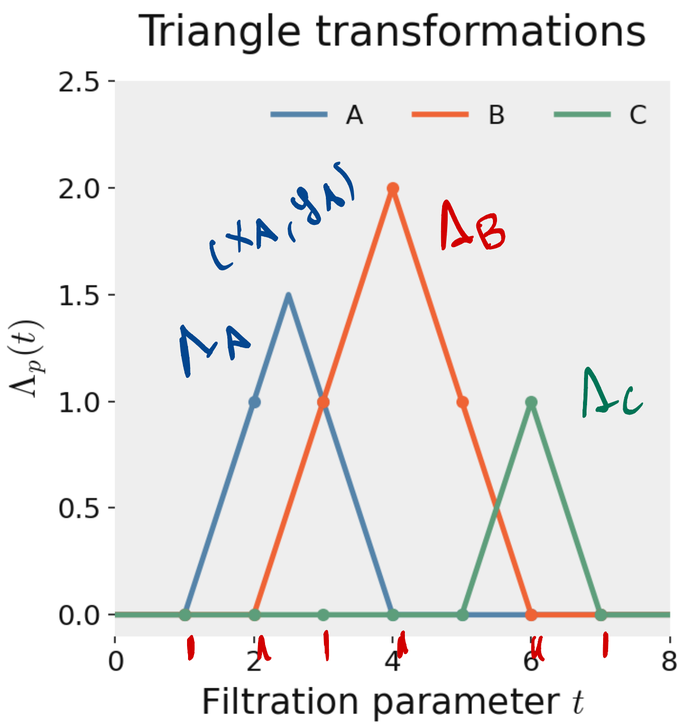
Triangle point transformation:

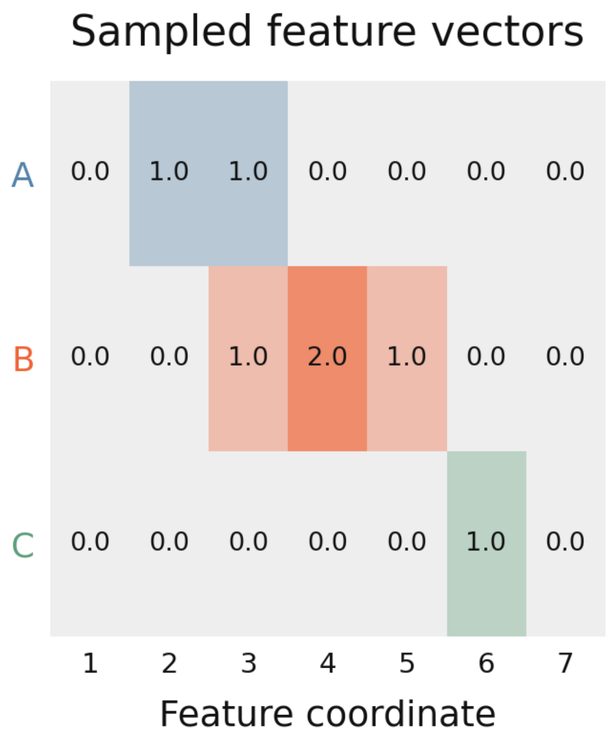
$$\phi_\Lambda : \mathbb{R}^2 \to \mathbb{R}^q, \qquad p = (x, y) \mapsto \big(\Lambda_p(t_1), \ldots, \Lambda_p(t_q)\big)$$

where $\Lambda_p : \mathbb{R} \to \mathbb{R},\ t \mapsto \max\{0,\ y - |t - x|\}$, for $q \in \mathbb{N},\ t_1, \ldots, t_q \in \mathbb{R}$ (learnable parameters)

$q = 7$







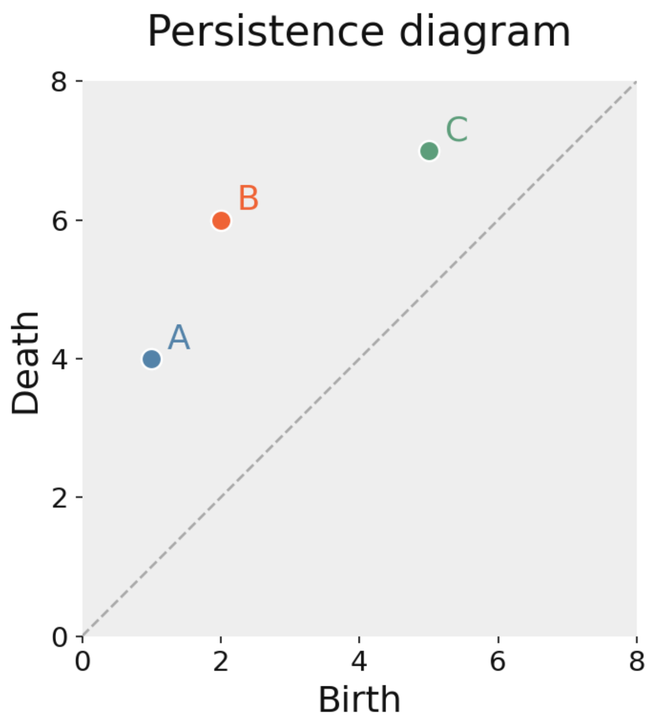
Point transformations

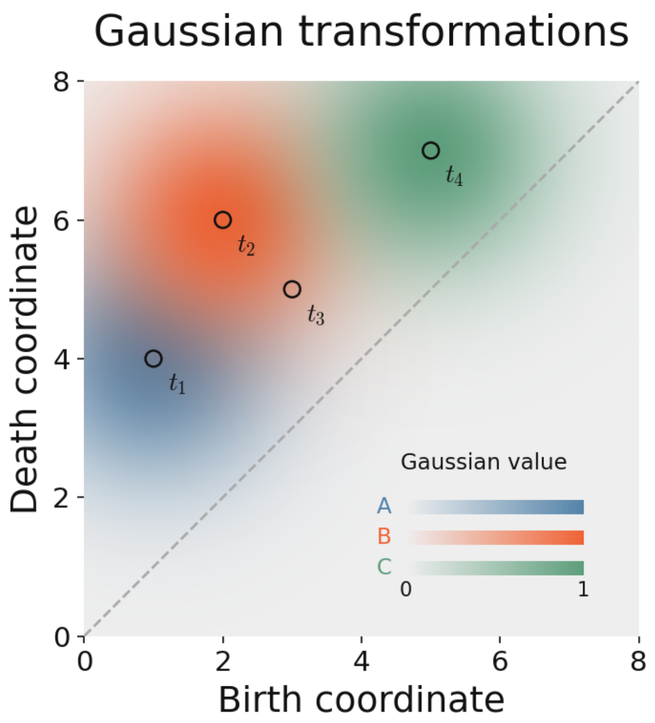
Gaussian point transformation:

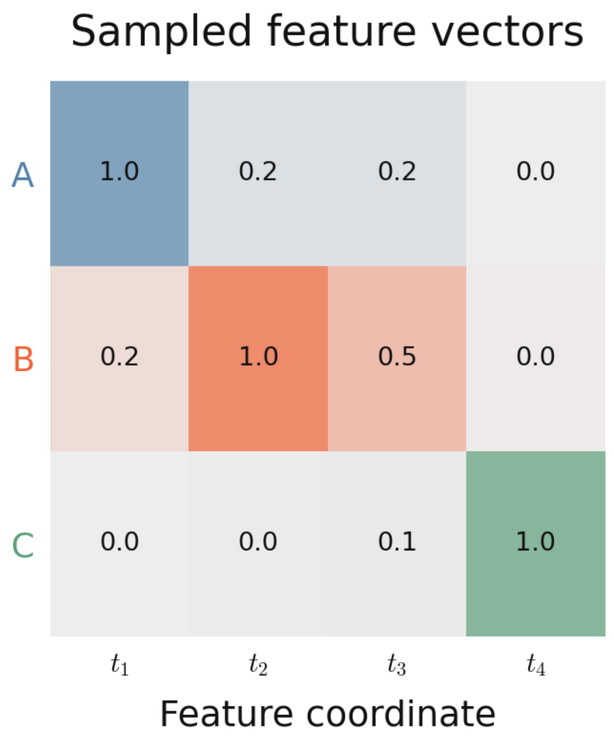
$$\phi_\Gamma : \mathbb{R}^2 \to \mathbb{R}^q, \qquad p = (x, y) \mapsto \big(\Gamma_p(t_1), \ldots, \Gamma_p(t_q)\big)$$

where $\Gamma_p : \mathbb{R}^2 \to \mathbb{R}, \quad t \mapsto \exp\big(\frac{-\|p-t\|^2}{2\sigma^2}\big)$, for $\sigma > 0$ (learnable parameter) and $q \in \mathbb{N}$, $t_1, \ldots, t_q \in \mathbb{R}^2$ (fixed)

$q = 4$







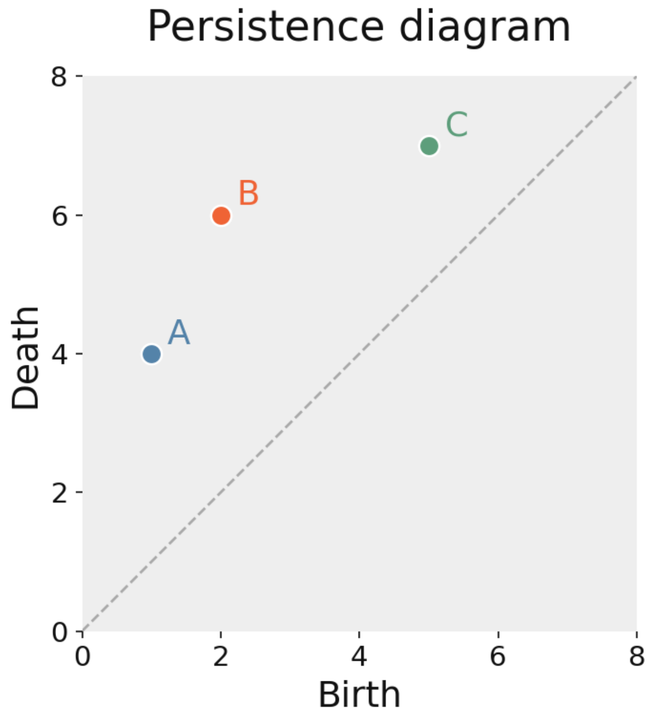
Point transformations

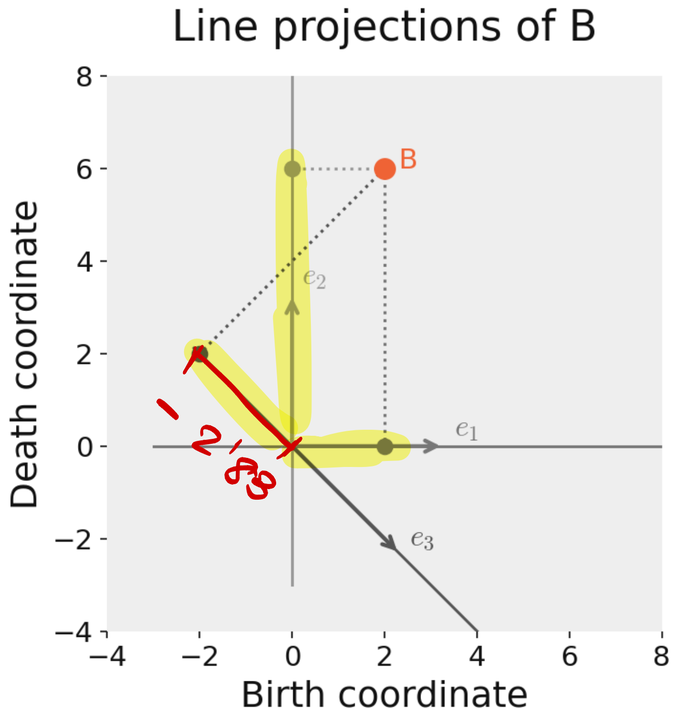
Line point transformation:

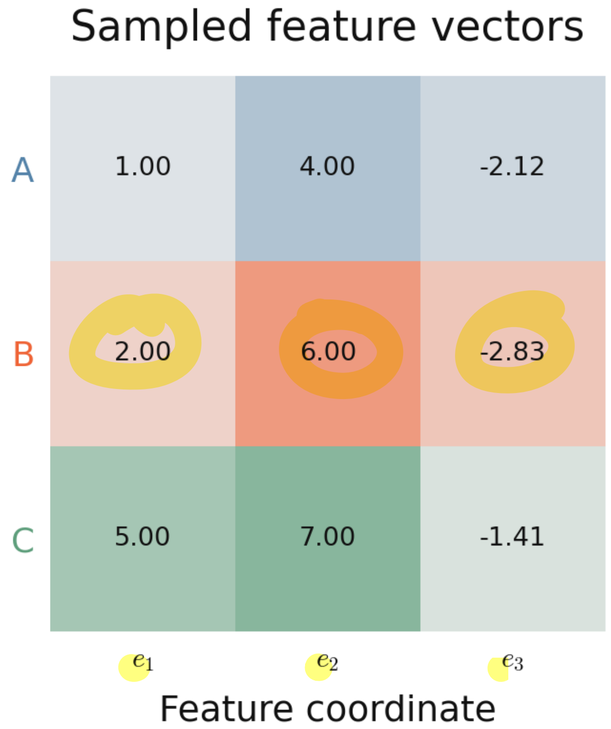
$$\phi_L : \mathbb{R}^2 \to \mathbb{R}^q, \qquad p = (x, y) \mapsto \big(L_{\Delta_1}(p), \ldots, L_{\Delta_q}(p)\big)$$

where $L_\Delta : \mathbb{R}^2 \to \mathbb{R}$, $p \mapsto \langle p, e_\Delta \rangle + b_\Delta$, for the line Δ with direction vector $e_\Delta \in \mathbb{R}^2$ and bias $b_\Delta \in \mathbb{R}$, $q \in \mathbb{N}$ and $\Delta_1, \ldots, \Delta_q$ are q lines on the plane

unit norm



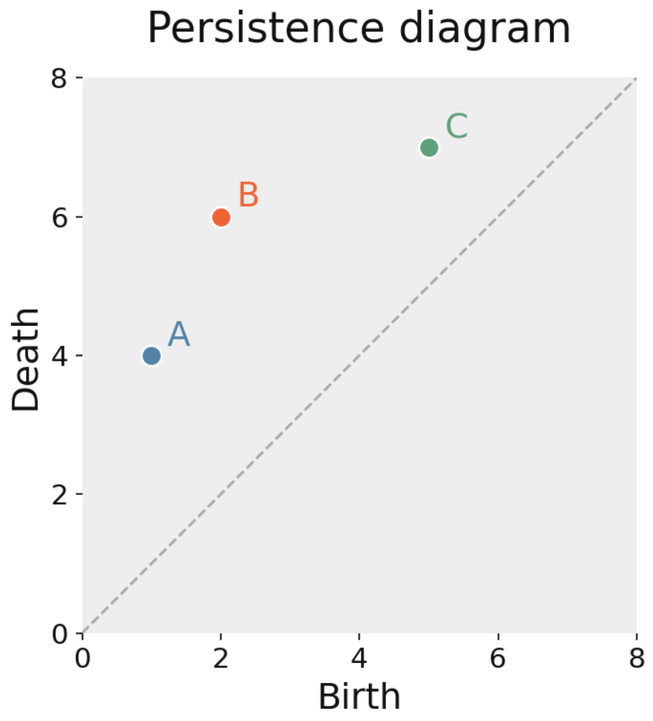


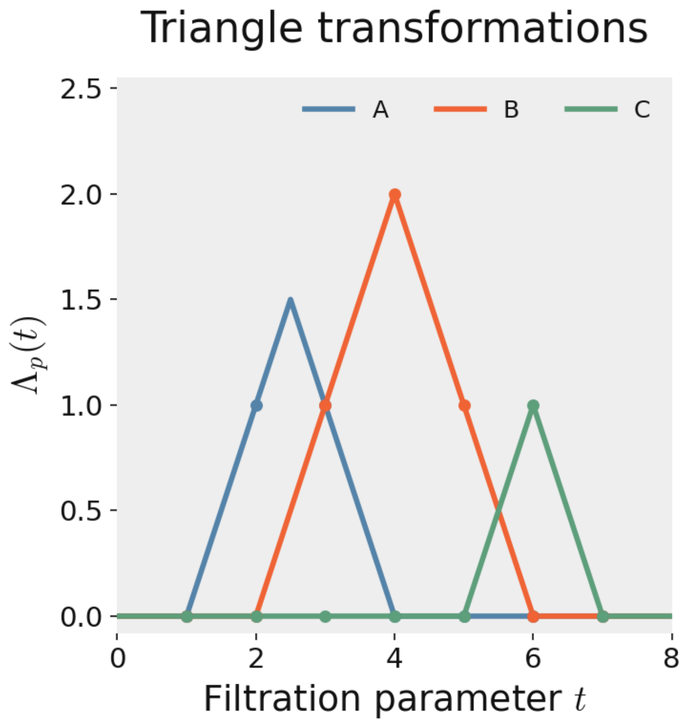


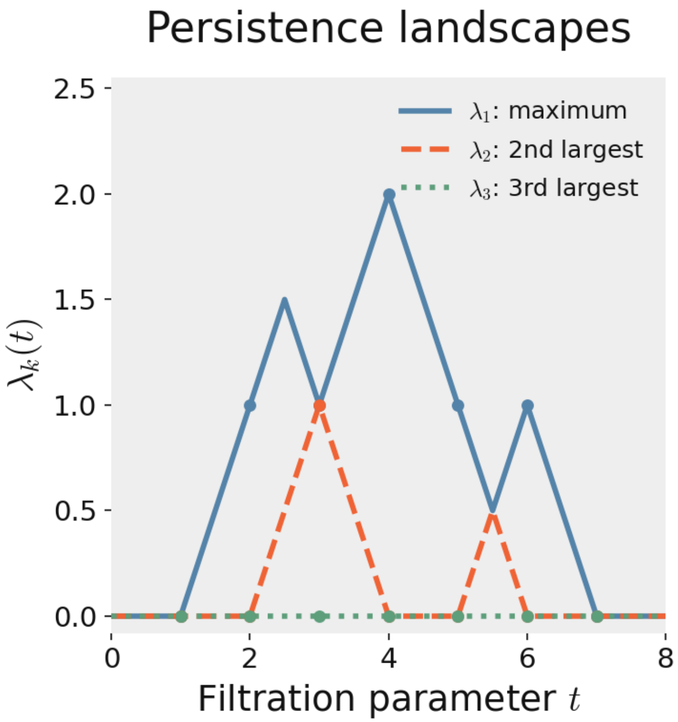
Recovering persistence landscapes

$$\phi = \phi_\Lambda \text{ (triangle point transformation)}, \quad w(p) = 1, \forall p, \quad \bigoplus = k\text{-th largest value per coordinate}$$

$$\left(\operatorname*{kmax}_{p \in \mathrm{PD}} \Lambda_p(t_1), \ldots, \operatorname*{kmax}_{p \in \mathrm{PD}} \Lambda_p(t_q) \right)\}$$







Application to graph classification

Input data: a graph $G = (V, E)$ seen as a discrete metric space (V, d) with V set of vertices in the graph and d a graph distance

Def. Shortest path distance

Let $G = (V, E)$ be an undirected graph, and $u, v \in V$ two vertices. A *path* between them is a collection of edges

$$\tau = \{e_i = (u_i, u_{i+1}) : 0 \leq i \leq n-1\}$$

with $u = u_0$ and $v = u_n$

The *length* of such a path is $n = \text{len}(\tau)$.

The *shortest path distance* between the vertices is defined as the minimal length among paths between them:

$$d_G : V \times V \to \mathbb{R}_{\geq 0} \qquad d_G(u, v) = \min_{\tau \text{ path}} \text{len}(\tau)$$

If $G = (V, E, w)$ $w : E \longrightarrow \mathbb{R}_{>0}$

$$\text{len}(\tau) = \sum_{e \in \tau} w(e)$$

Application to graph classification

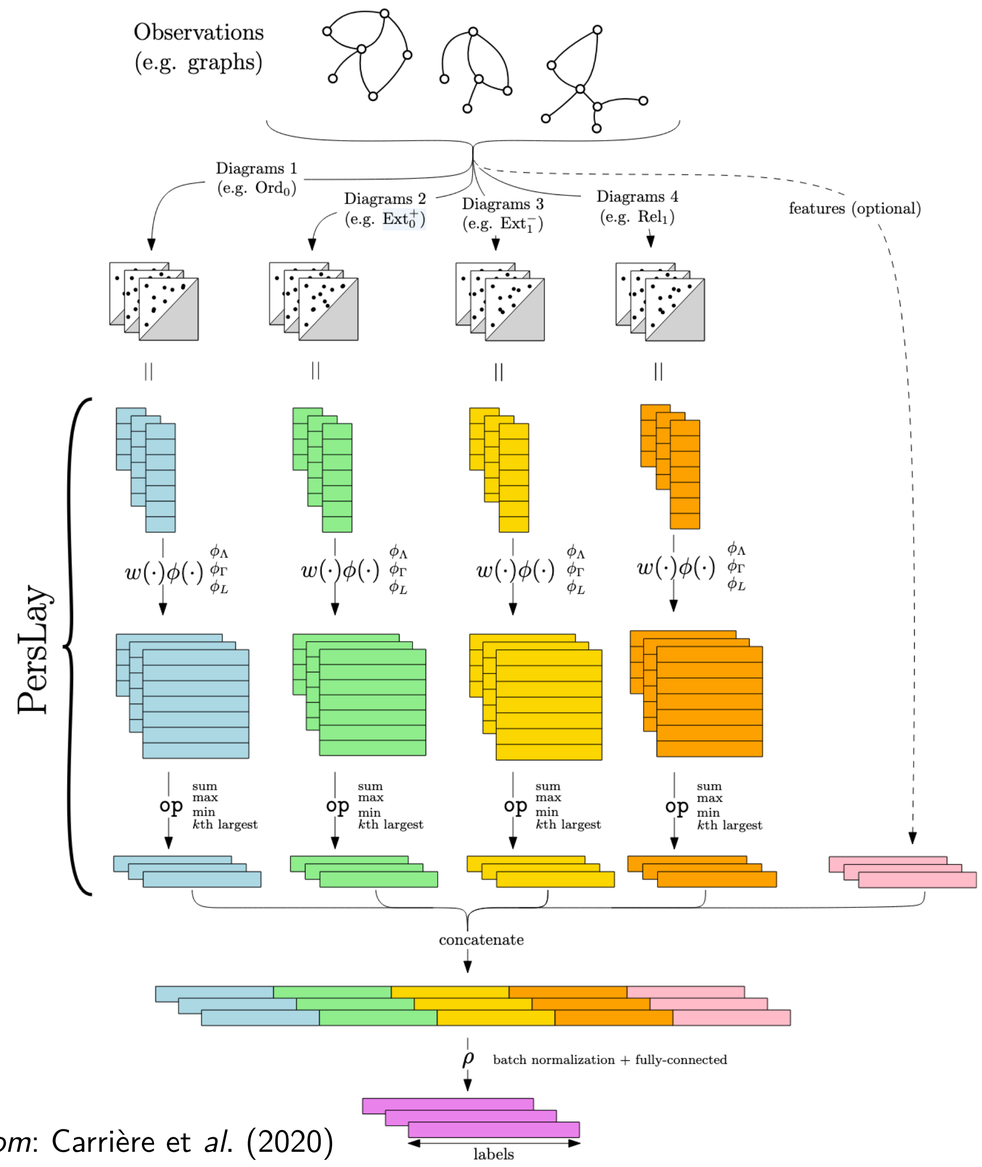


Figure from: Carrière et *al.* (2020)

Input data: graphs, seen as discrete metric spaces with a distance defined on the vertices according to the graph structure

Compute **persistence diagrams** from *extended persistence*

Apply the `PersLay` transformation for three point transformations and learnable weights

Concatenate all resulting vectors + normalize

Apply an **MLP** to classify

Datasets studied for the graph application

Dataset	Nb graphs	Nb classes	Av. nodes	Av. Edges	Av. β_0	Av. β_1
REDDIT5K	5,000	5	508.5	594.9	3.71	90.1
REDDIT12K	12,000	11	391.4	456.9	2.8	68.29
COLLAB	5,000	3	74.5	2457.5	1.0	2383.7
IMDB-B	1,000	2	19.77	96.53	1.0	77.76
IMDB-M	1,500	3	13.00	65.94	1.0	53.93
COX2	467	2	41.22	43.45	1.0	3.22
DHFR	756	2	42.43	44.54	1.0	3.12
MUTAG	188	2	17.93	19.79	1.0	2.86
PROTEINS	1,113	2	39.06	72.82	1.08	34.84
NCI1	4,110	2	29.87	32.30	1.19	3.62
NCI109	4,127	2	29.68	32.13	1.20	3.64

Social graphs

Molecular datasets

Table from: Carrière et *al.* (2020)

Results

Dataset	SV[1]	RetGK* [2]	FGSD [3]	GCNN [4]	GIN [5]	PersLay Mean	PersLay Max
REDDIT5K	—	56.1	47.8	52.9	57.0	55.6	56.5
REDDIT12K	—	48.7	—	46.6	—	47.7	49.1
COLLAB	—	81.0	80.0	79.6	80.1	76.4	78.0
IMDB-B	72.9	71.9	73.6	73.1	74.3	71.2	72.6
IMDB-M	50.3	47.7	52.4	50.3	52.1	48.8	52.2
COX2*	78.4	80.1	—	—	—	80.9	81.6
DHFR*	78.4	81.5	—	—	—	80.3	80.9
MUTAG*	88.3	90.3	92.1	86.7	89.0	89.8	91.5
PROTEINS*	72.6	75.8	73.4	76.3	75.9	74.8	75.9
NCI1*	71.6	84.5	79.8	78.4	82.7	73.5	74.0
NCI109*	70.5	—	78.8	—	—	69.5	70.1

Table from: Carrière et *al.* (2020)

- SV: kernel method on persistence diagrams for point clouds obtained from the graphs
- RetBK: graph kernel method using graph attributes
- FGSD: graph kernel method that doesn't use graph attributes
- GCNN, GIN: graph neural networks

Limitations

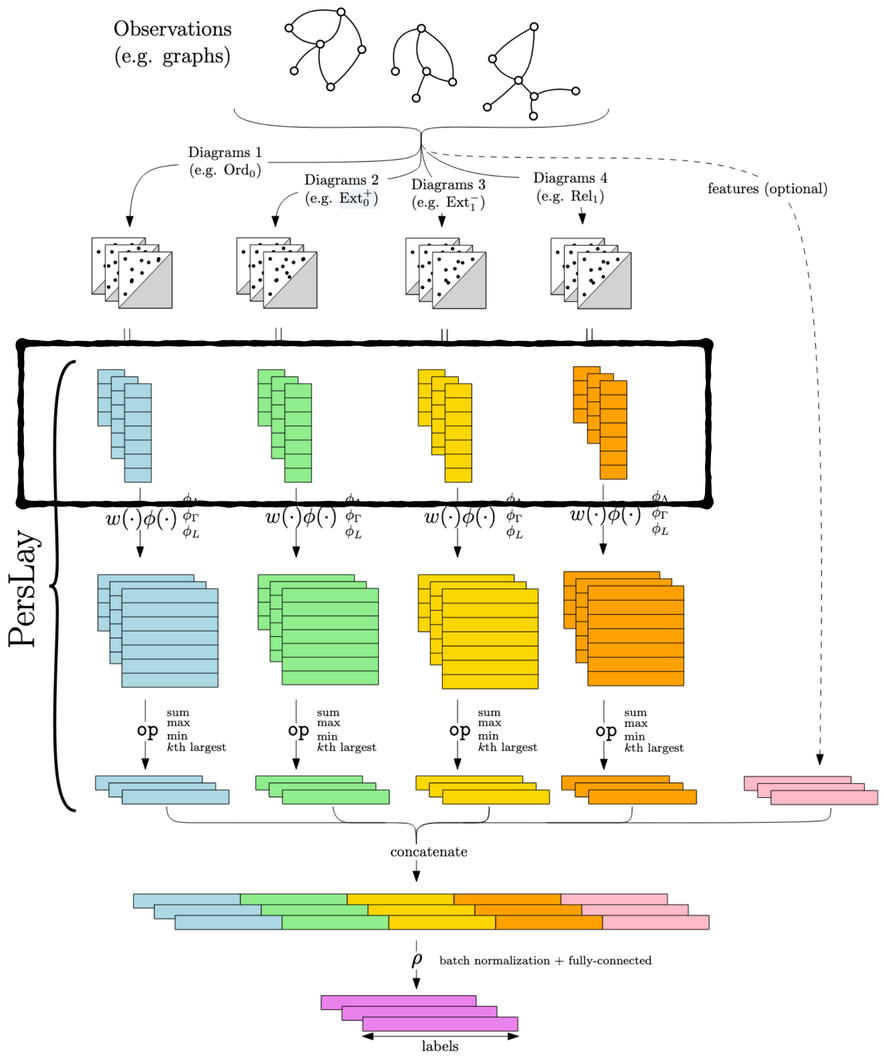


Figure from: Carrière et *al.* (2020)

Not learnable: the persistence computation is fixed, can we make it learnable?

Can we differentiate wrt any parameters here?

Learnable part: where the parameters that we are optimizing live

Topological losses: learning the filtration

Lecture 9 — Topological losses

Defining a loss on persistence diagrams

// $\mathrm{PD}(H_k(X_\bullet))$

Persistence diagram: $\mathrm{PD}_k = \{(b_i, d_i) : i \in \mathcal{I}_k\}$ $|\mathcal{I}_k|$ = # bars in the barcode of $H_k(X_\bullet)$

Order points by decreasing lifetime $d_i - b_i \geq d_j - b_j \iff i < j$

$i = 1 \to (b_0, d_0)$ longest bar in the barco

Goal: define a *differentiable* loss of these diagrams

fixed parameters of the loss — variables of the loss function ⇒ want it to be learnable parameters

$$\mathcal{L}(p, q, i_0; \mathrm{PD}_k) = \sum_{i=i_0}^{|\mathcal{I}_k|} |d_i - b_i|^p \left(\frac{d_i + b_i}{2}\right)^q$$

Leave the first i_0 most persistent features out

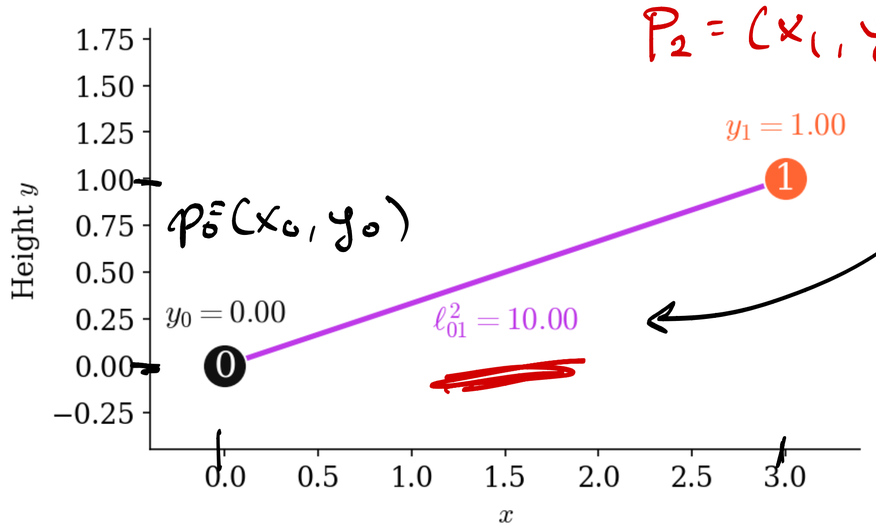
Increasing p penalizes more the most persistent features

Weight that gives more importance to bars appearing later in the filtration

Goal: we want to compute the derivative of the loss with respect to the birth and death values

Gabrielsson, R. B., Nelson, B. J., Dwaraknath, A., and Skraba, P. (2020). A Topology Layer for Machine Learning. *Proceedings of the Twenty Third International Conference on Artificial Intelligence and Statistics*, 1553–1563.

Computing the gradient of the loss: example



$$l_{01} = d(p_0, p_2)^2 = |x_0 - x_1|^2 + |y_0 - y_1|^2 = 9 + 1 = 10$$

Consider the following filtration over the point cloud:

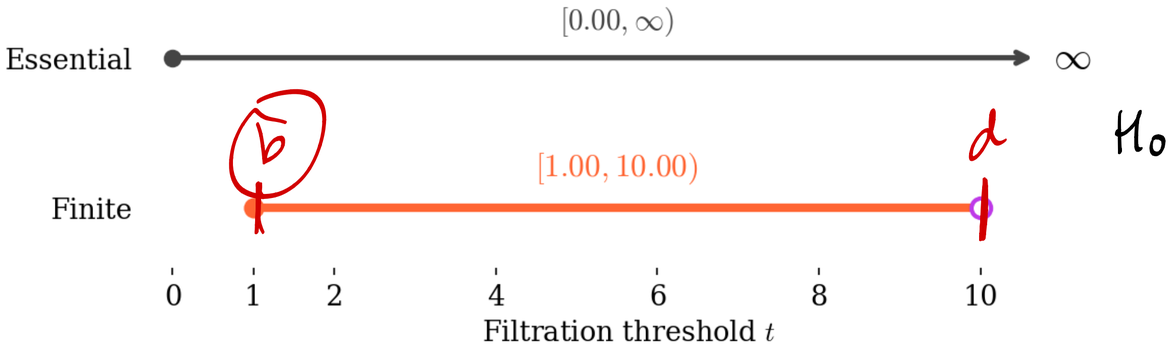
$$P = (p_0, p_1) \subset \mathbb{R}^2$$

- Points enter the filtration according to their height
- The edge enters the filtration according to the distance between the points

square

want to skip the essential bar

$$\mathcal{L}(2, 0, 2; \mathrm{PD}_0) = (d - b)^2$$



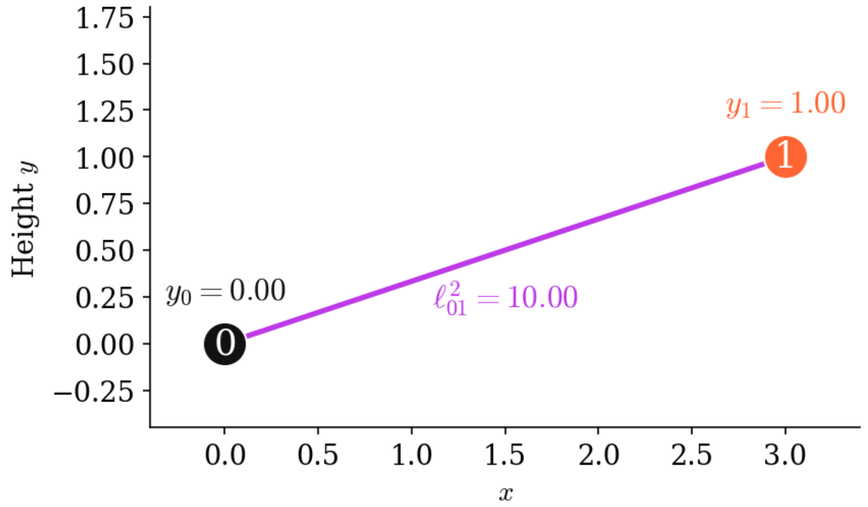
Try it: can you express the loss in terms of differentiable variables?

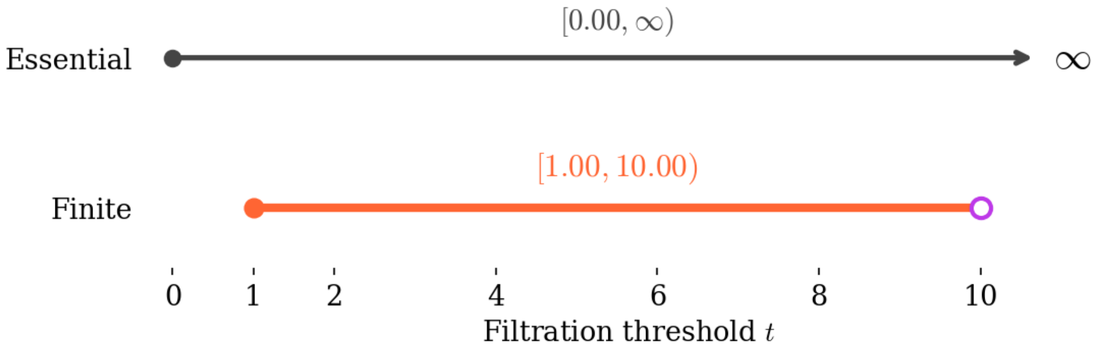
$b = y_1$

$d = |x_0 - x_1|^2 + |y_0 - y_1|^2$

$$\mathcal{L} = (|x_0 - x_1|^2 + |y_0 - y_1|^2 - y_1)^2$$

Computing the gradient of the loss: example





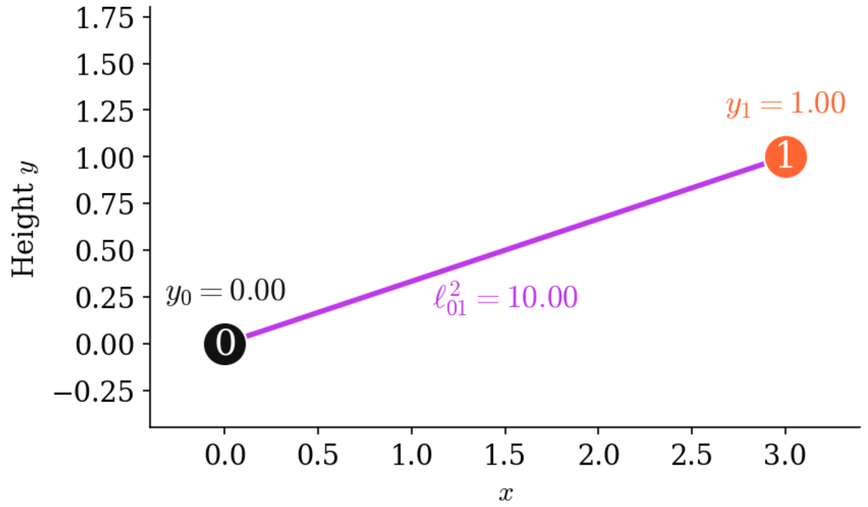
Try it: can you compute the derivatives of the loss wrt d and b?

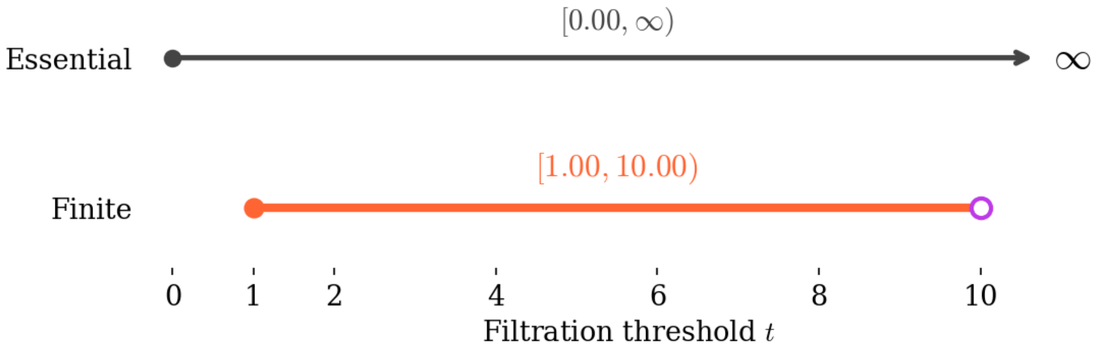
$$\frac{\partial \mathcal{L}}{\partial b} = -2(d-b)$$

$$\frac{\partial \mathcal{L}}{\partial d} = 2(d-b)$$

$$\mathcal{L} = (d-b)^2$$

Computing the gradient of the loss: example





Try it: can you use the previous computation to obtain the gradient of the loss wrt (x_0, x_1, y_0, y_1)?

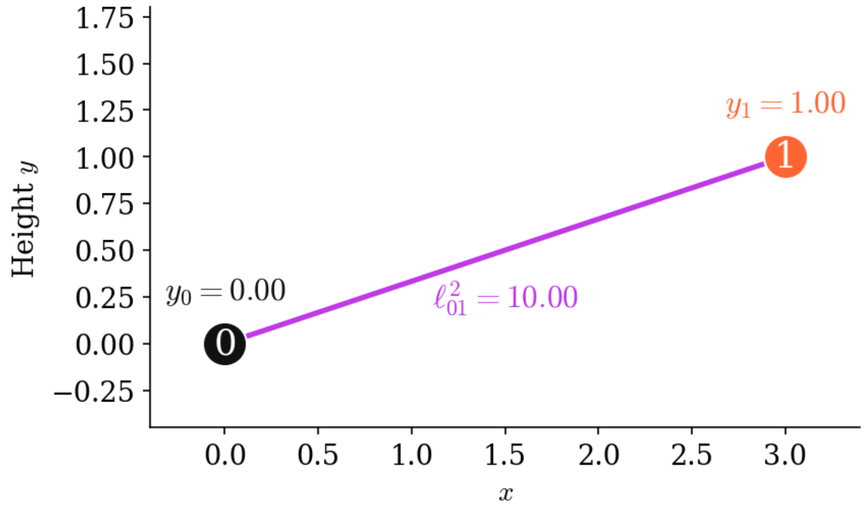
$$\nabla \ell = \frac{\partial \ell}{\partial b} \nabla b + \frac{\partial \ell}{\partial d} \nabla d$$

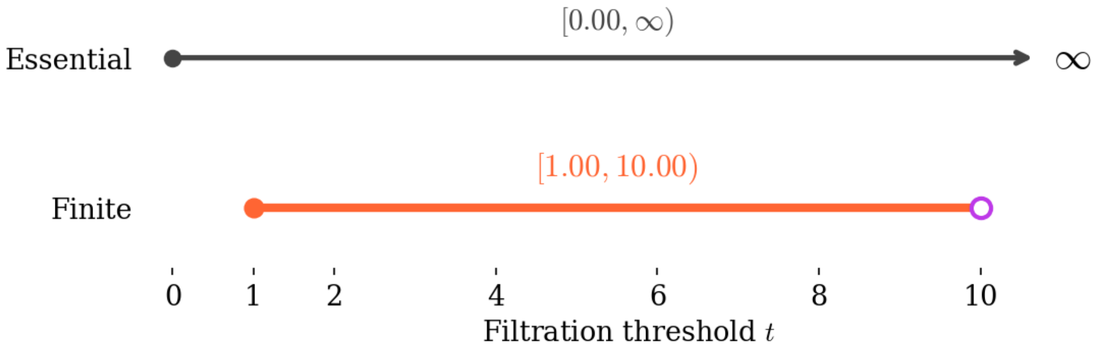
$$= -2r \begin{pmatrix} 0 \\ 0 \\ 0 \\ 1 \end{pmatrix} + 2r \begin{pmatrix} 2(x_0 - x_1) \\ -2(x_0 - x_1) \\ 2(y_0 - y_1) \\ -2(y_0 - y_1) \end{pmatrix}$$

$$r = (d - b)$$

$$\nabla \ell = \begin{pmatrix} 4r(x_0 - x_1) \\ 4r(x_1 - x_0) \\ 4r(y_0 - y_1) \\ 4r(y_1 - y_0) - 2r \end{pmatrix}$$

Computing the gradient of the loss: example





Try it: can you use evaluate that gradient on our given configuration?

$(x_0, y_0) = (0, 0)$

$(x_1, y_1) = (3, 1)$

$r = 10$

$$\nabla \mathcal{L}(p_0, p_1) = \begin{pmatrix} 4.9\ 3 \\ -4.9.3 \\ -4.9. \\ 4.9\ .1 - 2.9 \end{pmatrix}$$

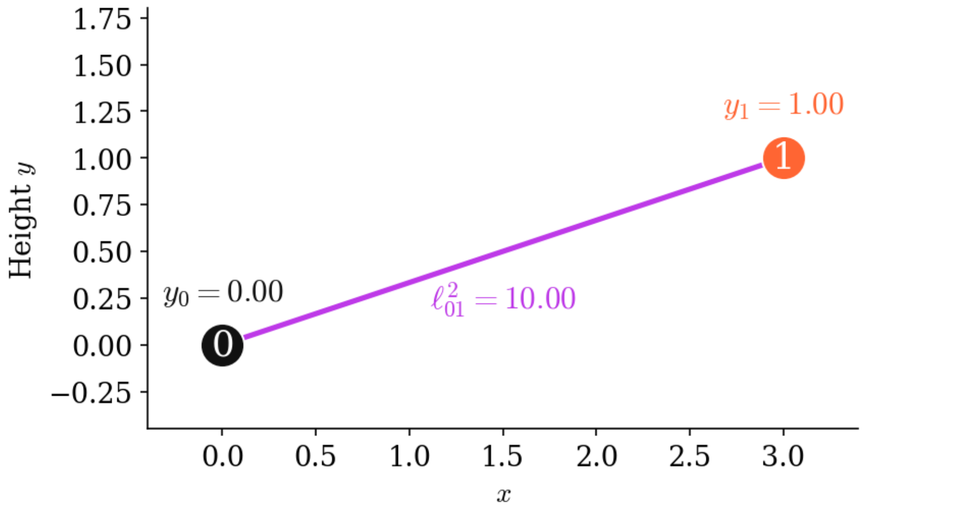
$$= \begin{pmatrix} 708 \\ -108 \\ -36 \\ 18 \end{pmatrix}$$

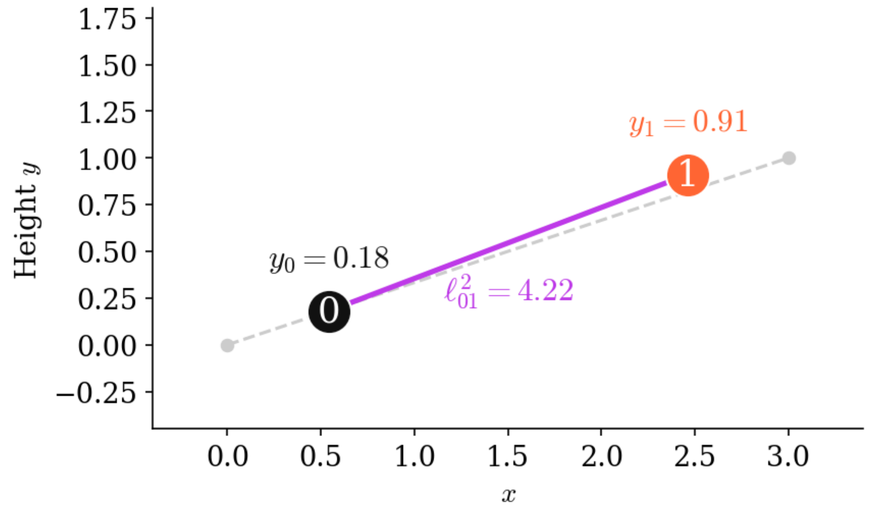
Computing the gradient of the loss: example

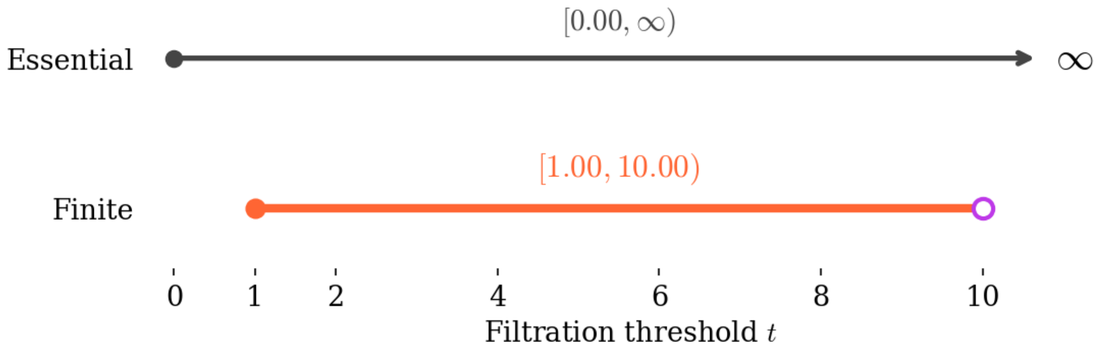
(x_0, x_1, y_0, y_1)

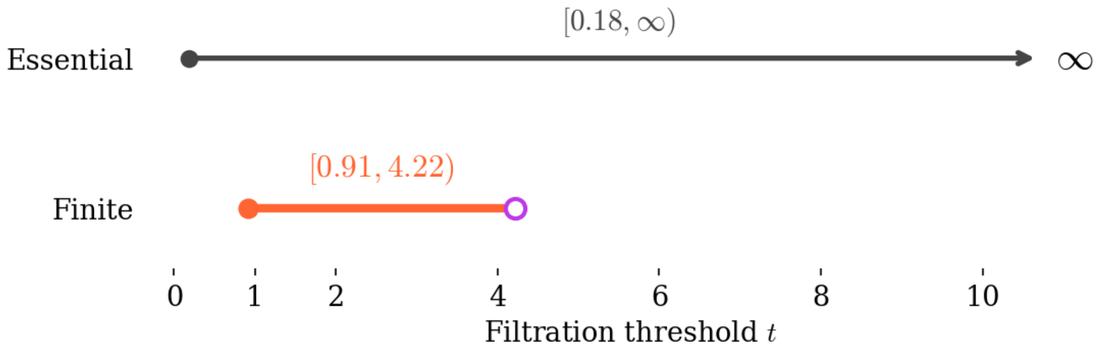
$$P_{\text{new}} \mapsto P - 0.005\, \nabla_P \mathcal{L}$$

$\mathcal{L} = (a-b)^2$









What did we need for this to work?

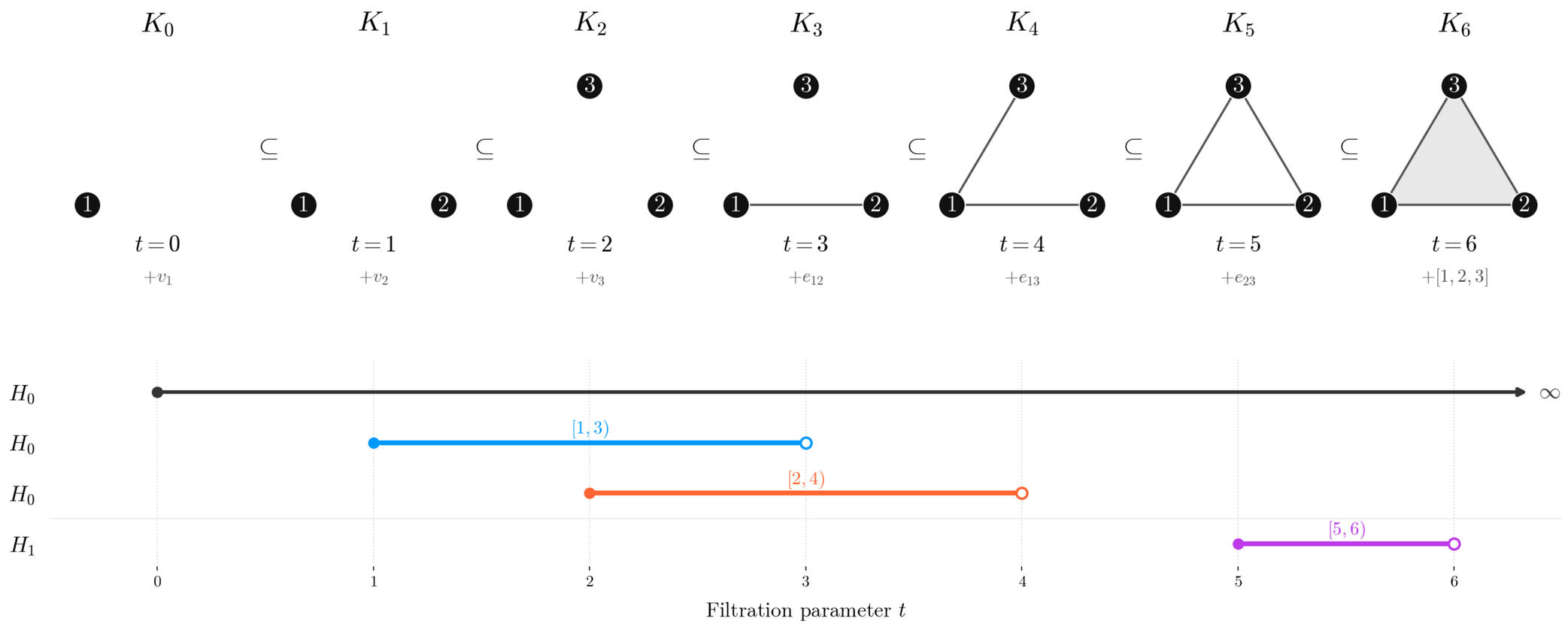
- We needed to match each birth and death time in a bar to a differentiable parameter given by the positions of the points
- In other words, we have matched each bar to the simplex that generated it (critical simplex)
- And then we have identified that simplex to its filtration value, obtained through differentiable position parameters

In the background, there is a big assumption that we can match uniquely bars to critical simplices, which is not true in general

Simplex-wise filtrations

Def. Simplex-wise filtration

A filtration $\{X_t : t \in \mathbb{R}\}$ such that at each parameter value $t \in \mathbb{R}$ at most one simplex enters the filtration



Can we extend to general filtrations?

Critical simplices: simplices that created and destroyed the topological feature represented by the bar

If the filtration is simplex-wise, we have a *unique map:*

$$\pi_k : \{(b_i, d_i) : i \in \mathcal{I}_k\} \to (\sigma, \tau)$$

In the background, there is a big assumption that we have a simplex-wise filtration which is not true in general

But we can always refine a filtration to make it simplexwise

Note: you can extend this to work with general filtrations, by making them simplex-wise choosing a total order of the simplices

In that case, the derivative should be considered as a *subgradient*, and choosing a total order selects an element in the subgradient

General point clouds

Figure from: Gabrielsson et *al.* (2020)

$$\mathcal{L}(2, 0, i_0; \mathrm{PD}_k) = \sum_{i=i_0}^{|\mathcal{I}_k|} |d_i - b_i|^2$$

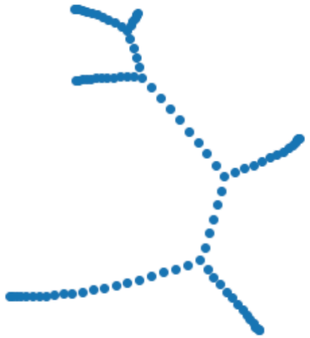
$\uparrow \mathcal{L}(2, 0, 2; \mathrm{PD}_0)$

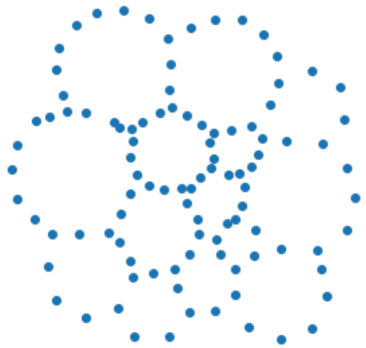
Uniform sample

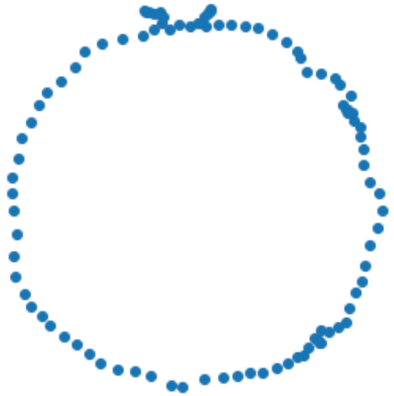
$\downarrow \mathcal{L}(2, 0, 2; \mathrm{PD}_0)$













$\uparrow \mathcal{L}(2, 0, 1; \mathrm{PD}_1)$

$\uparrow \mathcal{L}(2, 1, 1; \mathrm{PD}_1)$
$\downarrow \mathcal{L}(2, 0, 2; \mathrm{PD}_0)$

$\downarrow \mathcal{L}(2, 0, 1; \mathrm{PD}_1)$

Gabrielsson, R. B., Nelson, B. J., Dwaraknath, A., and Skraba, P. (2020). A Topology Layer for Machine Learning. *Proceedings of the Twenty Third International Conference on Artificial Intelligence and Statistics*, 1553–1563.

Thank you for your attention!



AIDOS LAB
AI FOR DATA-ORIENTED SCIENCE

UNI
FR

Lecture 9 — Topological losses

Inés García Redondo — November 10, 2026